

Билет 1

1. Классы и объекты в Python: определение класса, создание экземпляра, роль `self`. Атрибуты экземпляра и атрибуты класса.
2. Инкапсуляция в Python: защищённые (`_protected`) и приватные (`__private`) атрибуты. Управляемый доступ через декоратор `@property` (геттер, сеттер, делетер).

Билет 2

1. Конструктор `__init__`: назначение, параметры, момент вызова. Отличие атрибутов экземпляра от атрибутов класса.
2. Магические методы `__str__` и `__repr__`: различия в назначении, когда вызываются, правила реализации.

Билет 3

1. Наследование: синтаксис, переопределение методов, обращение к переопределённым методам через `super()`. Понятие порядка разрешения методов (MRO).
2. Статические методы и методы класса: `@staticmethod` и `@classmethod`, семантические различия, типичные сценарии использования.

Билет 4

1. Множественное наследование: проблема ромба, как Python её решает с помощью MRO. Кооперативное множественное наследование с `super()`.
2. Полиморфизм в Python: утиная типизация, переопределение методов, роль абстрактных базовых классов (`abc.ABC`, `@abstractmethod`).

Билет 5

1. Атрибуты класса и атрибуты экземпляра: способы объявления, поиск при чтении и особенности присваивания. Атрибут `__dict__`.
2. Декоратор `@property`: создание геттера, сеттера и делетера, преимущества перед явными методами доступа.

Билет 6

1. Методы экземпляра: как работает автоматическая передача `self`, понятие связанного метода (bound method). Вызов метода через класс с явной передачей экземпляра.
2. Магический метод `__call__`: как сделать экземпляр класса вызываемым. Примеры: функторы, классы-декораторы.

Билет 7

1. Примеси (mixins) в Python: идея, реализация через множественное наследование, отличия от обычных базовых классов.
2. Протокол итерации: методы `__iter__` и `__next__`. Создание собственного итератора. Сравнение итератора, реализованного через класс, и генератора на основе `yield`.

Билет 8

1. Магические методы сравнения: `__eq__` и `__hash__`. Их взаимосвязь и контракт. Когда требуется переопределять эти методы.
2. Менеджеры контекста: методы `__enter__` и `__exit__`. Реализация собственного менеджера контекста на основе класса.

Билет 9

1. Магический метод `__new__`: отличие от `__init__`, случаи переопределения (Singleton, наследование от неизменяемых типов).
2. Дескрипторы: протокол дескриптора (`__get__`, `__set__`, `__delete__`). Связь дескрипторов с работой декоратора `@property`.

Билет 10

1. Абстрактные классы: модуль `abc`, класс `ABC`, декоратор `@abstractmethod`. Как задать интерфейс и гарантировать его реализацию в подклассах.
2. Пользовательские исключения: создание классов исключений наследованием от `Exception`, построение иерархии исключений для приложения.

Билет 11

1. Структурная типизация и протоколы: `typing.Protocol`. Отличия от классического наследования и абстрактных классов, утиная типизация на уровне типов.
2. Эмуляция контейнеров: магические методы `__getitem__`, `__setitem__`, `__delitem__`, `__len__`. Создание собственного класса, подобного списку или словарю.

Билет 12

1. Функция `super()`: аргументы по умолчанию, работа в методах экземпляра, связь с MRO. Кооперативное наследование и корректная передача параметров.
2. Паттерн «Фабричный метод» с использованием `@classmethod`: создание альтернативных конструкторов.

Билет 13

1. Динамическая природа объектов в Python: добавление, изменение и удаление атрибутов во время выполнения. Функции `getattr`, `setattr`, `delattr`, `hasattr`.
2. Класс `object` как базовый класс всех классов: основные унаследованные методы и их поведение по умолчанию (`__str__`, `__repr__`, `__eq__`, `__hash__` и др.).

Билет 14

1. Утиная типизация: философия, примеры из стандартной библиотеки, сравнение с полиморфизмом на основе наследования.
2. Паттерн «Синглтон» в Python: реализация через переопределение `__new__`. Гарантия существования единственного экземпляра класса.

Билет 15

1. Деструктор `__del__`: когда вызывается, отличие от `__exit__` менеджера контекста, ограничения и рекомендуемые практики.

2. Обработка исключений в ООП: конструкция `try/except/else/finally`, перехват пользовательских исключений, проброс исключений.

Билет 16

1. Области видимости и пространства имён в классах: правило LEGB при поиске имени, доступ к атрибутам через `self` и через имя класса.
2. Сравнение `isinstance()` и `type()`: отличия в поведении, почему `isinstance()` предпочтительнее с учётом наследования и виртуальных подклассов.

Билет 17

1. Множественное наследование и примеси: построение кооперативных цепочек вызовов через `super()`, анализ MRO на конкретном примере.
2. Дескрипторы и методы класса: как функции, определённые в классе, становятся связанными методами; роль дескрипторного протокола в этом процессе.

Билет 18

1. Магический метод `__len__`: назначение, связь с проверкой объекта в булевом контексте, пример реализации.
2. Паттерн «Стратегия» в Python: реализация с помощью классов и с помощью функций, преимущества динамической типизации.

Билет 19

1. Хранение атрибутов: `__dict__` экземпляра и класса, разрешение имён при доступе к атрибутам. Динамическое добавление методов в класс и экземпляр.
2. Классы-декораторы: использование `__init__` и `__call__` для создания декоратора, сохраняющего состояние (например, подсчёт вызовов).

Билет 20

1. Итераторы и генераторы: реализация итератора через класс с методами `__iter__`/`__next__` и через функцию-генератор с `yield`. Преимущества и недостатки каждого подхода.
2. Сравнение абстрактных классов (`ABC`) и протоколов (`Protocol`): когда использовать наследование от `ABC`, а когда – структурную типизацию.

Билет 21

1. Композиция и агрегация как альтернатива наследованию. Принцип «предпочитайте композицию наследованию» применительно к Python.
2. Виды дескрипторов: data descriptor (`__get__`, `__set__`/`__delete__`) и non-data descriptor (`__get__` только). Как это влияет на приоритет при поиске атрибута.

Билет 22

1. Метод `__new__` и паттерн Singleton: подробный разбор реализации, контроль количества экземпляров.

2. Пользовательские исключения: создание цепочки исключений (`raise ... from ...`), сохранение контекста исходной ошибки.

Билет 23

1. Переопределение `__eq__` и `__hash__`: контракт между равенством и хешированием, почему изменяемые объекты обычно не должны определять `__hash__`.
2. Класс `object`: методы `__eq__`, `__hash__`, `__repr__`, `__str__` по умолчанию, что они возвращают и как их переопределение влияет на поведение объектов.

Билет 24

1. Использование `@classmethod` в качестве альтернативного конструктора: примеры (`from_json`, `from_csv`), преимущества перед обычными методами.
2. Магический метод `__del__` и корректное управление ресурсами: сравнение с менеджерами контекста, гарантии вызова.

Билет 25

1. Наследование и композиция в проектировании классов на Python: критерии выбора, примеры.
2. Паттерн «Стратегия»: детальная реализация с динамической подстановкой поведения, сравнение классов-стратегий и функций-стратегий.